

ui.dropdown_button 全面详解（基于 NiceGUI 文档）

ui.dropdown_button 是 NiceGUI 中用于创建带下拉菜单的复合按钮组件，基于 Quasar 的 QBtnDropDown 组件实现。它融合了普通按钮的触发特性与下拉菜单的选项扩展能力，支持自定义菜单内容、样式定制和灵活的交互控制，适用于需要收纳多个关联操作的界面场景。以下从核心特性、基础用法、高级功能等维度展开全面解析。

一、核心基础

1. 组件本质与核心特性

- 底层依赖：基于 Quasar 的 QBtnDropDown 组件，继承其成熟的下拉交互逻辑和样式体系。
- 颜色兼容性：支持 Quasar 颜色、Tailwind 颜色和 CSS 颜色，优先级为「Quasar 颜色 > CSS 颜色」（同名颜色如 "red" 优先使用 Quasar 配色）。
- 灵活扩展性：下拉菜单内可嵌套任意 NiceGUI 组件（如按钮、开关、输入框等），而非局限于固定选项。

2. 初始化参数（核心配置）

参数名	类型	说明	默认值
text	str	按钮的文本标签	-
value	bool	下拉菜单是否默认展开（True = 展开，False = 收起）	False
on_value_change	Callable	下拉菜单展开 / 收起状态变化时触发的回调函数	-
on_click	Callable	按钮本身被点击时触发的回调函数（2.22.0+ 版本支持）	-
color	str / None	按钮颜色（Quasar/Tailwind/CSS 颜色）	'primary'
icon	str / None	按钮上显示的图标名称（符合 Quasar 图标规范）	None
auto_close	bool	点击下拉菜单内的元素后，是否自动关闭菜单	False
split	bool	是否将下拉图标拆分为独立按钮（主按钮触发 on_click，拆分按钮控制菜单展开 / 收起）	False

二、基础使用示例

1. 最简下拉按钮（默认交互）

包含基础文本按钮和下拉选项，点击选项触发通知，设置 auto_close=True 点击后自动关闭菜单：

```

from nicegui import ui

# 下拉菜单点击选项后自动关闭
with ui.dropdown_button('Open me!', auto_close=True):
    ui.item('Item 1', on_click=lambda: ui.notify('You clicked item 1'))
    ui.item('Item 2', on_click=lambda: ui.notify('You clicked item 2'))
    ui.item('Item 3', on_click=lambda: ui.notify('You clicked item 3'))

ui.run()

```

2. 带图标 + 拆分按钮的下拉菜单

通过 `icon` 参数添加按钮图标，`split=True` 拆分下拉控制按钮，主按钮与拆分按钮各司其职：

```

from nicegui import ui

# 拆分模式：主按钮触发点击事件，右侧小按钮控制下拉菜单
with ui.dropdown_button('Settings', icon='settings', split=True, on_click=lambda:
    ui.notify('Main button clicked!')):
    ui.item('Profile', on_click=lambda: ui.notify('Edit Profile'))
    ui.item('Notifications', on_click=lambda: ui.notify('Manage Notifications'))
    ui.item('Logout', on_click=lambda: ui.notify('Logout'))

ui.run()

```

效果：主按钮显示“Settings”文本 + 设置图标，点击触发 `on_click` 回调；右侧小按钮仅控制下拉菜单的展开 / 收起。

3. 自定义下拉菜单内容（非选项类元素）

下拉菜单内可嵌套任意组件（如开关、图标、分隔符），实现复杂交互界面：

```

from nicegui import ui

with ui.dropdown_button('Controls', icon='tune', auto_close=False):
    # 嵌套行布局，包含图标、开关和垂直分隔符
    with ui.row().classes('p-4 items-center gap-4'):
        ui.icon('volume_up', size='sm')
        ui.switch('Volume').props('color=blue')

        ui.separator().props('vertical') # 垂直分隔符

        ui.icon('mic', size='sm')
        ui.switch('Microphone').props('color=red')

    # 新增独立选项
    ui.item('Reset Settings', on_click=lambda: ui.notify('Settings reset!'))

ui.run()

```

效果：下拉菜单包含开关控制项和普通选项，`auto_close=False` 点击开关后菜单不关闭，方便连续操作。

三、核心功能与进阶用法

1. 下拉菜单状态监听 (on_value_change)

通过 `on_value_change` 回调监听菜单展开 / 收起状态，可用于联动其他界面元素：

```
from nicegui import ui

# 监听下拉菜单的展开/收起状态
def on_menu_toggle(e):
    state = 'opened' if e.value else 'closed'
    ui.notify(f'Dropdown menu {state}')

with ui.dropdown_button('Toggle Me', on_value_change=on_menu_toggle):
    ui.item('Option A')
    ui.item('Option B')

ui.run()
```

效果：展开菜单时触发 `e.value=True`，收起时触发 `e.value=False`，回调函数实时反馈状态。

2. 手动控制下拉菜单 (open/close/toggle)

通过组件实例的 `open()`、`close()`、`toggle()` 方法，手动控制菜单的展开 / 收起：

```
from nicegui import ui

# 创建下拉按钮并赋值给变量，用于手动控制
dropdown = ui.dropdown_button('Controllable Dropdown')
with dropdown:
    ui.item('Item 1')
    ui.item('Item 2')

# 手动控制按钮
with ui.row().classes('mt-4'):
    ui.button('Open', on_click=dropdown.open)
    ui.button('Close', on_click=dropdown.close)
    ui.button('Toggle', on_click=dropdown.toggle)

ui.run()
```

效果：点击“Open”强制展开菜单，“Close”强制收起，“Toggle”切换当前状态。

3. 数据绑定 (动态控制属性)

通过 `bind_*` 系列方法，将按钮的文本、图标、状态等与数据对象绑定，实现动态更新：

```
from nicegui import ui

class AppState:
    def __init__(self):
```

```

        self.button_text = 'Dynamic Menu'
        self.button_icon = 'menu'
        self.is_enabled = True

state = AppState()

# 绑定文本、图标和启用状态
dropdown = ui.dropdown_button(
    text=state.button_text,
    icon=state.button_icon,
).bind_text(state, 'button_text') \
    .bind_icon(state, 'button_icon') \
    .bind_enabled(state, 'is_enabled')

with dropdown:
    ui.item('Change Text', on_click=lambda: setattr(state, 'button_text', 'Updated Menu'))
    ui.item('Change Icon', on_click=lambda: setattr(state, 'button_icon', 'star'))
    ui.item('Disable Button', on_click=lambda: setattr(state, 'is_enabled', False))

ui.run()

```

效果：点击下拉选项可动态修改按钮的文本、图标，或禁用按钮，无需手动调用 `update()`。

4. 样式定制（颜色、类与 props）

支持通过 `color`、`classes`、`props` 自定义按钮样式，与普通 `ui.button` 样式逻辑一致：

```

from nicegui import ui

# 自定义颜色、圆角和轮廓样式
with ui.dropdown_button(
    'Styled Button',
    color='purple-600', # Tailwind 颜色
    icon='magic',
).props('rounded outline') \ # Quasar props: 圆角+轮廓样式
    .classes('shadow-md px-4 py-2'): # Tailwind 类: 阴影+内边距
    ui.item('Option 1').classes('text-purple-800')
    ui.item('Option 2').classes('text-purple-800')
    ui.separator() # 水平分隔符
    ui.item('Delete', classes('text-red-600'))

ui.run()

```

效果：按钮为紫色轮廓样式，带阴影和内边距，下拉选项区分普通选项和危险操作选项。

四、核心属性与方法

1. 常用属性

属性名	类型	说明
classes	Classes[Self]	按钮的 CSS 类（支持 Tailwind/Quasar 类）
enabled	BindableProperty	按钮是否启用（可绑定数据）
html_id	str	HTML 元素的 ID（2.16.0+ 版本支持）
icon	BindableProperty	按钮图标（可绑定数据动态修改）
text	BindableProperty	按钮文本（可绑定数据动态修改）
value	BindableProperty	菜单展开状态（True = 展开，False = 收起，可绑定数据）
visible	BindableProperty	按钮是否可见（可绑定数据）
props	Props[Self]	Quasar 特性属性（如 rounded、outline、glossy 等）

2. 关键方法

(1) 状态与交互控制

- `open()`：展开下拉菜单
- `close()`：收起下拉菜单
- `toggle()`：切换下拉菜单的展开 / 收起状态
- `disable()`：禁用按钮（菜单无法展开）
- `enable()`：启用按钮
- `set_enabled(value: bool)`：设置启用状态（True/False）
- `set_visibility(visible: bool)`：设置按钮可见性

(2) 属性修改

- `set_text(text: str)`：动态修改按钮文本
- `set_icon(icon: str | None)`：动态修改按钮图标
- `set_value(value: bool)`：设置菜单展开状态（True/False）
- `clear()`：移除下拉菜单内的所有子元素
- `update()`：触发客户端界面更新（修改属性后需手动调用，绑定数据时无需）

(3) 事件与绑定

- `on_click(callback)`：绑定按钮点击事件（2.22.0+ 版本）
- `on_value_change(callback)`：绑定菜单展开 / 收起状态变化事件
- `on(type: str, handler)`：订阅任意 DOM 事件（如 `mousedown`、`mouseover`）

- `bind_text(target_object, target_name)`: 双向绑定文本到目标对象属性
- `bind_value_from(target_object, target_name)`: 单向绑定菜单状态从目标对象
- `bind_visibility(target_object, target_name)`: 双向绑定可见性

(4) 其他实用方法

- `tooltip(text: str)`: 为按钮添加悬停提示
- `delete()`: 删除按钮及所有子元素（下拉菜单）
- `remove(element)`: 移除下拉菜单内的指定子元素
- `mark(*markers)`: 添加标记（用于测试或元素查询）

五、使用场景与注意事项

1. 适用场景

- 操作收纳: 将多个关联但非核心的操作（如“编辑、删除、分享”）收纳在下拉菜单中，简化界面。
- 配置面板: 下拉菜单内嵌套开关、输入框等组件，实现轻量化配置界面（如音量、权限设置）。
- 复合交互: 通过 `split` 模式拆分主按钮与下拉控制，主按钮触发核心操作，下拉菜单提供次要选项。

2. 关键注意事项

- 版本兼容性: `on_click` 方法仅在 2.22.0+ 版本支持，`html_id` 仅在 2.16.0+ 版本支持，使用时需确认版本。
- `auto_close` 默认为 `False`: 默认情况下点击下拉菜单内的元素不会关闭菜单，需手动设置 `auto_close=True` 实现“点击即关闭”。
- `split` 模式的事件逻辑: `split=True` 时，主按钮触发 `on_click` 回调，拆分按钮仅控制菜单展开 / 收起，不会触发 `on_click`。
- 下拉菜单层级: 下拉菜单默认在按钮下方展开，若页面底部空间不足，会自动向上调整位置（基于 Quasar 自动定位逻辑）。
- 样式一致性: 若需在 `ui.button_group` 中使用 `ui.dropdown_button`，需确保父组件与子组件的 `props` 一致（如同时设置 `outline`），避免样式错乱。

六、进阶示例：带搜索功能的下拉菜单

结合 `ui.input` 实现下拉菜单内的选项搜索，提升多选项场景的可用性：

```
from nicegui import ui

# 模拟选项数据
options = ['Apple', 'Banana', 'Cherry', 'Date', 'Grape', 'Mango', 'Orange']

with ui.dropdown_button('Searchable Menu', icon='search', auto_close=False):
    # 搜索输入框
    search_input = ui.input(placeholder='Search...').classes('mb-2')

    # 选项容器（用于动态更新选项）
    options_container = ui.column()
```

```
# 过滤选项的函数
def filter_options(e):
    query = e.value.lower() if e.value else ''
    options_container.clear()
    for item in options:
        if query in item.lower():
            ui.item(item, on_click=lambda x=item: ui.notify(f'Selected:
{x}')).parent(options_container)

# 绑定搜索输入框的变化事件
search_input.on('input', filter_options)

# 初始加载所有选项
filter_options({'value': ''})

ui.run()
```

效果：下拉菜单内包含搜索框，输入关键词可实时过滤选项，点击选项触发通知，菜单保持展开状态方便连续搜索。